

AEGIS-ECO

Event-Driven Resource Orchestration Platform
For Conservation, Heritage & Scientific Field Projects

Technical Design & Architecture Report
Version 1.0 | 2026

Executive Summary

AEGIS-ECO is a domain-driven, event-sourced distributed resource orchestration engine designed to coordinate the complex lifecycle of conservation surveys, heritage restoration projects, and scientific field expeditions. Unlike conventional project management tools, AEGIS-ECO treats resource scheduling, regulatory compliance, and funding allocation as first-class distributed systems problems.

The platform is built around the insight that long-running government and scientific field projects — which routinely span weeks or months before active fieldwork can begin — require asynchronous, compensating-transaction-aware orchestration that traditional synchronous software fundamentally cannot provide.

Core Problem Statement

How do you coordinate funding, permits, domain experts, laboratories, drones, field equipment, and survey teams across long-running government and scientific projects while handling partial failures gracefully and maintaining full auditability?

Core Philosophy

Domain Pain First — Architecture Necessity Second

Most distributed systems projects begin with a technology selection and retrofit a domain onto it. AEGIS-ECO inverts this pattern deliberately:

Conventional Approach	AEGIS-ECO Approach
1. Choose Kafka 2. Choose a domain 3. Retrofit the problem	1. Identify domain pain 2. Map failure modes 3. Architecture emerges

Why This Domain Earns Every Technology Choice

Each architectural decision in AEGIS-ECO has a direct, explainable domain justification:

- Kafka is used because a forest clearance permit can take 5 days or 5 weeks — no database transaction can remain open that long
- Saga pattern is used because partial failure (permit rejected after funds and equipment are already reserved) requires structured compensation
- PostGIS is used because drone availability is meaningless without knowing its distance from the project site
- Redis is used because equipment availability dashboards must respond in milliseconds, not wait on complex joins
- Outbox pattern is used because updating the database and publishing a Kafka event are not atomic operations

Domain Analysis

Target Operational Contexts

AEGIS-ECO addresses a specific and currently underserved intersection of operational domains:

Conservation	Heritage	Scientific Survey
Western Ghats eDNA surveys Wetland biodiversity mapping National park corridor studies Satellite wildlife tracking	ASI archaeological digs Heritage site restoration Ground-penetrating radar surveys UNESCO documentation projects	Biodiversity baseline studies Geophysical mapping expeditions Remote sensing campaigns Soil and water quality surveys

Competitive Landscape

Existing software solutions address fragments of this problem space. No existing platform combines all required capabilities:

Platform	Strengths	Gaps	Why AEGIS-ECO Differs
GBIF / iNaturalist	Species records, observations	No resource orchestration	Adds scheduling and permit workflows
Esri / QGIS	Spatial analysis, mapping	No distributed workflow	Integrates GIS into booking decisions
Jira / MS Project	Tasks, timelines, teams	No permits, no equipment reservations	Saga-aware long-running workflows
IBM Maximo / SAP EAM	Asset tracking, maintenance	Extremely expensive and rigid	Open, domain-specific, cloud-native

System Architecture & Methodology

The Saga Orchestration Pattern

The Saga pattern is the conceptual and technical heart of AEGIS-ECO. It replaces distributed two-phase commit (2PC) with a sequence of local transactions, each publishing an event that triggers the next step. When a step fails, compensating transactions undo all prior steps.

Why Not 2PC?

Two-phase commit requires all participants to lock resources for the duration of the transaction. When permit approval takes 15 days, holding a database lock for 15 days is not a strategy — it is a disaster. The Saga pattern was designed precisely for this scenario.

Happy Path Saga Flow

1. ProjectCreated — Project Service publishes creation event
2. FundsReserved — Funding Service allocates grant budget
3. ResourcesReserved — Resource Service books nearest available equipment
4. TeamAssigned — Team Service assigns researchers and field staff
5. PermitRequested — Regulatory Gateway submits clearance applications
6. PermitApproved — All clearances received (may take days to weeks)
7. ProjectActivated — Field operations can now begin

Failure & Compensation Flow

When a permit is rejected at step 5, the following compensation cascade executes automatically:

1. PermitRejected — Regulatory Gateway publishes rejection event
2. ReleaseResources — Resource Service frees all reserved equipment
3. ReleaseFunds — Funding Service returns budget allocation
4. UnassignTeam — Team Service frees all assigned personnel
5. ProjectCancelled — Project Service marks project as cancelled with full audit trail

The GIS Dimension

The geographic awareness layer transforms AEGIS-ECO from a resource tracker into a resource optimization engine. Equipment availability without location context is incomplete information.

Without GIS	With PostGIS
Drone #3: Available No further context.	Drone #3: Available · Pune depot Project site: Satara · Distance: 110 km Travel time: 4 hours · Battery: suitable Decision: RESERVE

Microservice Design

Service Responsibilities

AEGIS-ECO Version 1 ships four focused services. Each service owns its database, communicates exclusively via Kafka events, and implements the Outbox pattern for guaranteed message delivery.

1. Project Service

Saga orchestrator and system brain

- Creates and tracks project lifecycle
- Publishes ProjectCreated event to initiate Saga
- Monitors compensation state across services
- Statuses: DRAFT, PENDING_RESOURCES, PENDING_PERMITS, ACTIVE, COMPLETED, CANCELLED

2. Resource Service

Scarce equipment reservation engine

- Manages exclusive physical assets (drones, sequencers, GPS units)
- Implements soft → hard → deployed → released reservation lifecycle
- Spatial query: find nearest available asset within radius
- Compensates via ReleaseResources on Saga failure

3. Regulatory Gateway

Multi-clearance permit management

- Orchestrates Forest, Archaeological, Environmental, and Drone permits
- Handles PENDING → APPROVED / REJECTED / EXPIRED / REQUIRES_REVISION states
- Naturally asynchronous: permits arrive days or weeks after request

4. Funding Service

Grant budget allocation and recovery

- Tracks grants, reserved amounts, and expenditure per project
- Produces FundsReserved on ProjectCreated
- Compensates via FundsReleased on any upstream failure
- Immutable ledger model — every transaction is an event

- Publishes PermitApproved or PermitRejected to trigger Saga continuation

The Outbox Pattern

Every service implements the Transactional Outbox Pattern to eliminate the dual-write problem — the risk that a service updates its database but fails before publishing the corresponding Kafka event (or vice versa).

The Dual-Write Problem

UPDATE projects SET status = 'ACTIVE'; then PUBLISH to Kafka — what happens if the process crashes between these two steps? Without the Outbox, you get inconsistent state with no recovery path. With the Outbox, the database commit IS the publish.

Phase	Action
1. Application	Update domain table AND insert into outbox table in same DB transaction
2. Commit	Both writes commit atomically — the Kafka event is now guaranteed
3. Worker	Background Outbox Worker polls the outbox table for pending events
4. Publish	Worker publishes to Kafka and marks outbox row as PUBLISHED
5. Recovery	On crash, unpublished rows are retried automatically on restart

Technology Stack

Core Technology Choices & Justifications

Technology	Component	Domain Justification
Drogon Framework	HTTP/REST API layer	Modern async C++ with native PostgreSQL support and production-grade performance — no framework overhead
Apache Kafka + librdkafka	Message backbone	Permit workflows span weeks; only an async event bus can handle workflows that outlast any connection
PostgreSQL + PostGIS	Primary data store	ACID transactions for the Outbox pattern; PostGIS for spatial queries (nearest available equipment)
Redis (v2)	Dashboard cache	Equipment heatmaps and availability dashboards require sub-millisecond reads that would overwhelm PostgreSQL
Docker / Compose	Containerization	Reproducible local environment for all 4 services + Kafka + PostgreSQL + Redis

Kafka Topic Schema

Topic	Published By	Consumed By
project-created	Project Service	Funding, Resource, Team, Regulatory
funds-reserved	Funding Service	Project Service (Saga tracker)
resource-reserved	Resource Service	Project Service (Saga tracker)
team-assigned	Team Service	Project Service (Saga tracker)
permit-requested	Regulatory Gateway	External systems / Audit log
permit-approved	Regulatory Gateway	Project Service → ProjectActivated
permit-rejected	Regulatory Gateway	Project Service → compensation cascade
project-cancelled	Project Service	All services → release resources

Implementation Roadmap

Phase	Deliverables	Architecture Added
V1 Foundation	Project Service, Resource Service, Funding Service, Regulatory Gateway. Core Saga with compensation. Docker Compose environment.	Kafka, PostgreSQL (per-service DBs), Outbox pattern, Saga orchestration, PostGIS spatial queries
V2 Observability	Analytics Service, Equipment availability dashboard, Funding utilization dashboard, District-level project heatmap.	Redis cache layer, read-model projections, dashboard event consumers
V3 Auditability	Event Sourcing store, project timeline reconstruction, full audit replay, CQRS read models.	Event store, replay engine, CQRS separation of read/write models, event versioning

Critical Caution on Scope

Do not attempt to build Kubernetes, Terraform, Prometheus, Grafana, CQRS, and Event Sourcing simultaneously with V1. Students who do this spend 8 months debugging YAML instead of writing software. Build V1 with 4 services. Make the Saga work. Make compensation work. Then expand.

Project Structure

```

AegisEco/
  project-service/  → Saga brain, project lifecycle
  resource-service/ → Equipment booking + PostGIS queries
  funding-service/  → Grant allocation and release
  regulatory-service/ → Permit workflows
  shared/           → Kafka schemas, common utilities
  docker-compose.yml → Full environment definition
  docs/            → Architecture diagrams, API specs

```

Why This Project Stands Apart

The rarest and most valuable attribute of AEGIS-ECO is that every engineering decision answers a domain question — not a textbook question. When an interviewer asks "why Kafka?", the answer is not "because it is scalable." The answer is:

The Answer That Matters

Because a forest clearance permit from the Karnataka Forest Department can take three weeks. You cannot hold a database transaction open for three weeks. The entire project workflow must be asynchronous, with durable state and compensating transactions — and that is exactly what Kafka and the Saga pattern provide.

This is the difference between a project that demonstrates knowledge of distributed systems and one that demonstrates understanding of why distributed systems exist.

The combination of PostGIS, Kafka, Saga compensation, conservation domain knowledge, and spatial resource scheduling represents an intersection that most engineers — at any level — have never been asked to explain. That is what makes AEGIS-ECO memorable.

Enhancement Areas: Making AEGIS-ECO Interview-Ready

The following ten enhancements elevate AEGIS-ECO from a scheduling engine into a genuine decision-support platform. Each addresses a gap that experienced interviewers will probe — and each has a direct domain justification.

1. Resource Optimization Engine

The current resource reservation logic selects the nearest available drone. That is scheduling, not optimization. When an interviewer asks "why this drone?", the answer "because it was closest" reveals a gap. A true optimization engine scores each candidate resource across multiple weighted dimensions and selects the best overall fit.

Composite Scoring Formula

$$\text{score} = w1 \cdot \text{distance} + w2 \cdot \text{travel_time} + w3 \cdot \text{battery_capacity} + w4 \cdot \text{current_workload} + w5 \cdot \text{maintenance_schedule} + w6 \cdot \text{weather_conditions} + w7 \cdot \text{project_priority}$$

Example: Drone A is 50 km away with 20% battery. Drone B is 90 km away with 90% battery. The optimizer selects Drone B — proximity alone would have made the wrong call. This is the difference between a reservation engine and a decision engine.

2. Real GIS Intelligence with Spatial Constraints

The current PostGIS integration finds the nearest resource. That is useful but not impressive. The upgrade is to enforce spatial constraints so that resource allocation must navigate geographic restrictions — making PostGIS central to every deployment decision rather than decorative.

Spatial Constraint Zones

- Protected Forest Zone — drone entry prohibited
- No-Fly Zone — airspace restriction polygon
- Monsoon Flood Risk Zone — seasonal overlay
- Wildlife Sensitivity Zone — restricted access period

Upgraded Query Logic

Find nearest drone WHERE route does not intersect any active constraint zone. The resource allocation algorithm must now solve a constrained spatial problem, not just a proximity query. PostGIS becomes the decision layer, not a lookup table.

3. Priority-Based Scheduling

When two projects compete for a single drone — a high-priority Tiger Census and a low-priority Wetland Mapping survey — the system must make a principled decision. Without a priority model, the platform allocates to whichever request arrived first. That is a queue, not a scheduler. Introduce project priority as a first-class field (CRITICAL, HIGH, MEDIUM, LOW) and build the resource reservation logic to prefer higher-priority projects under contention. This is where engineering gets genuinely interesting.

4. Multi-Project Conflict Resolution

V1 implicitly assumes a single project context. Production reality is Project A, Project B, and Project C simultaneously competing for Drone #7. This introduces three classic distributed systems problems that make excellent interview material: resource starvation (low-priority projects never get allocated), priority queues (how to order competing reservations fairly), and reservation conflicts (what happens when two Sagas claim the same resource at the same moment). Adding multi-project conflict resolution transforms the Resource Service from a simple booking engine into a proper concurrency-aware scheduler.

5. Realistic Failure Modes

The current compensation flow handles PermitRejected cleanly. Real field deployments fail in messier ways. Three high-value failure scenarios to add:

Equipment Failure

Drone crashes mid-deployment. The Saga must detect a DroneFailedMidDeployment event, trigger compensation for the active project, and reallocate to the next best candidate — all without human intervention.

Permit Expiration

Permit approved, project delayed 30 days, permit expires before deployment begins. The Regulatory Gateway must emit a PermitExpired event and the Saga must decide whether to re-apply or cancel.

Funding Withdrawal

Government budget cut revokes a previously reserved grant mid-project. Funds reserved → FundsRevoked event → full Saga compensation cascade including releasing equipment and notifying the team. This is realistic for government-funded conservation work.

6. Event Versioning and Schema Evolution

V3 introduces Event Sourcing, but the report does not address what happens when an event's schema changes. In a live Kafka system, ProjectCreated v1 events may coexist in the same topic with ProjectCreated v2 events. Consumers must handle both. Schema evolution strategies — backward compatibility, forward compatibility, and full compatibility — are a real production concern. Mentioning schema registries (Confluent Schema Registry with Avro or Protobuf) and envelope versioning (adding a version field to every event payload) instantly raises the maturity signal of the architecture section.

7. Idempotency and Duplicate Event Handling

This is the most critical distributed systems gap in the current design. Kafka delivers at-least-once semantics by default — a consumer can and will receive the same event more than once. Without idempotency, a FundsReserved event processed twice reserves the grant budget twice. The forest project now has double its intended allocation.

Idempotency Pattern

Every event carries a globally unique Event ID. Each consumer maintains a Deduplication Table. Before processing any event, the consumer checks whether that Event ID has already been processed. If yes, skip. If no, process and record. This guarantees that FundsReserved is applied exactly once regardless of how many times Kafka delivers it. Idempotent consumers are a non-negotiable requirement in any production event-driven system and interviewers with distributed systems experience will ask about this directly.

8. Observability and Distributed Tracing

Nobody teaches observability in college. Everyone asks about it in industry. In a distributed system with four services and an asynchronous event bus, when something goes wrong — and it will — you need to reconstruct the full chain of events across all services for a single project. Three identifiers make this possible:

Correlation ID

Propagated through every event header. Lets you query logs across all four services for a single request chain.

Saga ID

Identifies which Saga instance an event belongs to. Lets the Project Service track compensation progress across multiple concurrent Sagas.

Trace ID	OpenTelemetry-compatible. Enables end-to-end timing of the entire Saga from ProjectCreated to ProjectActivated, including per-step latency breakdown. Essential for understanding where delays actually occur.
-----------------	--

9. User Personas

The report identifies Conservation, Heritage, and Scientific Survey as operational contexts. These are domains, not users. A platform without human actors feels like architecture searching for a problem. Define the four primary personas who interact with AEGIS-ECO:

Forest Officer	Submits project requests, tracks permit status, monitors active deployments within their jurisdiction. Primary concern: regulatory compliance and timeline.
Research Coordinator	Plans multi-site survey campaigns, coordinates team availability, tracks equipment reservations across concurrent projects. Primary concern: resource availability and scheduling conflicts.
NGO Program Manager	Oversees grant utilization, monitors project health across their portfolio, flags projects at risk of budget or permit failure. Primary concern: donor accountability and project success rate.
Lab Administrator	Manages sequencers, centrifuges, and specialist equipment. Tracks maintenance windows, approves reservations against maintenance schedules, flags equipment approaching service intervals. Primary concern: equipment uptime and calibration integrity.

10. The Killer Feature: Smart Deployment Planner

Every memorable project has one feature that non-engineers understand immediately. Currently, Kafka, Saga, and PostGIS are architecture features — powerful, but invisible to users. The Smart Deployment Planner translates the entire technical stack into a single user-facing capability that anyone can explain.

User Provides • Project location (coordinates) • Required equipment list • Required team composition • Budget ceiling • Target deployment deadline	System Returns • Optimal resource allocation plan • Estimated total cost • Estimated travel time to site • Permit requirements and lead times • Risk score (spatial + regulatory + resource)
---	---

With the Smart Deployment Planner, AEGIS-ECO evolves from an orchestration platform into a decision support system. The positioning becomes: not just “we coordinate your resources” but “we tell you the best way to deploy before you commit.” That is a meaningfully stronger product story — and a much more interesting engineering problem.

Future Scope

I. System Topology & Predictive Flow

In this model, your Analytics Service is promoted to the Risk Prediction Engine. It runs asynchronously, pulling telemetry data from your Kafka message bus, calculating scores, and updating a specialized Risk Read-Model inside Redis for real-world decision support.

II. The Core Telemetry Matrix

Instead of utilizing heavy machine-learning black boxes, you calculate structural risk metrics natively in your C++ backend using historical transaction sets, spatial proximity via PostGIS, and simple statistical weights.

Risk Category	Input Parameters (PostgreSQL Tables)	Computational Metric	Operational Impact
Regulatory Risk	Historical clearance_time per state/district office inside regulatory_db.	Moving average + Standard Deviation of regulatory agency approval intervals.	Flags projects likely to stall in a PENDING_PERMITS state for more than 14 days.
Resource Risk	Deployment hours, age, and maintenance logs inside resource_db.	Linear degradation curve mapping device age against average failure rates.	Rejects soft-reservations on hardware assets showing high critical failure probabilities.
Logistics Risk	PostGIS travel calculations and real-time external API weather data streams.	Matrix intersecting distance with weather severity ratings.	Calculates physical transit bottlenecks (e.g., transport windows during heavy monsoon seasons).

Risk Category	Input Parameters (PostgreSQL Tables)	Computational Metric	Operational Impact
Financial Risk	Micro-transfer velocity and historical budget adjustment tracking logs.	Variance between early-stage actual burn rates vs. initially allocated grants.	Flags projects trending toward early capital depletion.